# Docker Swarm em produção: deploy sem downtime e rollback

Material informativo · conferido na documentação oficial do Docker em 24/10/2026

O Docker Swarm transforma uma ou mais máquinas em um cluster que roda **serviços** com réplicas, atualização gradual e volta automática quando algo dá errado. Este guia mostra o essencial para publicar e atualizar um serviço sem derrubá-lo. Se a sua versão do Docker for diferente, a documentação oficial vale mais do que este guia.

## 1. Conceitos em um minuto

| Termo | O que é |
|---|---|
| **Nó** | Uma máquina no cluster. Pode ser *manager* (decide) ou *worker* (executa) |
| **Serviço** | O que você quer rodando: imagem, réplicas, portas, rede |
| **Tarefa** | Um container que o Swarm cria para cumprir o serviço |
| **Stack** | Um conjunto de serviços descritos em um arquivo Compose e publicados juntos |

## 2. Começar

```
docker swarm init                       # transforma esta máquina em manager
docker stack deploy -c stack.yml app    # publica a stack chamada "app"
docker service ls                       # lista os serviços
docker service ps app_web               # mostra as tarefas de um serviço
```

## 3. Um stack.yml com atualização e volta automáticas

```
services:
  web:
    image: nginx:1.27
    ports:
      - "8080:80"
    healthcheck:
      test: ["CMD", "wget", "-qO-", "http://localhost/"]
      interval: 10s
      timeout: 3s
      retries: 3
    deploy:
      replicas: 3
      update_config:
        parallelism: 1          # uma réplica por vez
        delay: 10s              # espera entre uma e outra
        order: start-first      # sobe a nova antes de derrubar a antiga
        failure_action: rollback
        monitor: 30s            # observa cada tarefa depois de atualizar
      rollback_config:
        parallelism: 1
        order: stop-first
      restart_policy:
        condition: any
      resources:
        limits:
          memory: 256M
```

Pontos que fazem diferença:

- `healthcheck`: sem ele, o Swarm só sabe que o processo subiu, não que a aplicação responde.
- `order: start-first`: evita o intervalo sem réplica. Exige que a aplicação aguente duas versões ao mesmo tempo, então cuide de migrações de banco compatíveis.
- `failure_action: rollback`: se a atualização falhar, o Swarm volta sozinho para a versão anterior.
- **Fixe a versão da imagem.** `latest` impede o rollback de saber o que era "a anterior".

## 4. Atualizar sem downtime

```
docker service update --image nginx:1.27.1 app_web
docker service inspect --pretty app_web     # política de atualização e estado
docker service ps app_web                   # progresso tarefa por tarefa
```

Se a atualização pausar ( `Update status: paused` ), corrija a causa e retome com `docker service update app_web` .

Flags úteis do `docker service update` :

| Flag | Para que serve |
|---|---|
| `--update-parallelism N` | Quantas tarefas atualizar ao mesmo tempo ( `0` = todas) |
| `--update-delay T` | Espera entre as atualizações, como `10s` |
| `--update-order start-first` | Sobe a nova tarefa antes de parar a antiga |
| `--update-failure-action` | `pause` , `continue` ou `rollback` quando falha |
| `--update-monitor T` | Quanto tempo vigiar cada tarefa depois de atualizar |
| `--force` | Recria as tarefas sem mudar nada (reinício gradual) |

## 5. Voltar atrás

```
docker service update --rollback app_web
```

Isso devolve o serviço à especificação anterior. Dá para ajustar a velocidade da volta com as flags `--rollback-*` .

## 6. Cuidados em produção

- **Banco de dados e volumes:** um volume local fica no nó onde foi criado. Se a tarefa mudar de nó, os dados não vão junto. Para serviços com estado, fixe o nó com `placement.constraints` ou use armazenamento compartilhado.
- **Segredos:** use `docker secret` em vez de variáveis de ambiente com senhas dentro do arquivo.
- **Redes:** deixe os serviços em redes internas e publique só o necessário.
- **Managers:** use um número ímpar (3 ou 5) para manter o quórum. Um manager só é um ponto único de falha.
- **Recursos:** defina limites de memória e CPU para um serviço não consumir o nó inteiro.

## 7. Checklist antes de publicar

- [ ] A imagem tem versão fixa, e não `latest` ?
- [ ] Há `healthcheck` que testa a aplicação de verdade?
- [ ] `update_config` tem `failure_action: rollback` e `monitor` definidos?
- [ ] Você ensaiou o rollback pelo menos uma vez?
- [ ] Os serviços com estado têm volume e nó definidos, e há backup dos dados?
- [ ] As senhas estão em secrets e não no arquivo?

- [ ] Há limite de memória e CPU nos serviços?

## 8. Problemas comuns

| Sintoma | O que verificar |
|---|---|
| A atualização pausa sozinha | A nova tarefa falhou o healthcheck ou caiu. Veja `docker service ps --no-trunc` e os logs |
| Réplicas ficam em `Pending` | Falta recurso ou nenhum nó atende às restrições de placement |
| A tarefa reinicia em loop | A aplicação sai com erro. Veja `docker service logs` |
| O dado "sumiu" depois de um deploy | A tarefa foi para outro nó e o volume local ficou no anterior |
| Rollback não volta ao esperado | A imagem anterior usava a tag `latest`, que já foi sobrescrita. Fixe versões |

## 9. Referências oficiais

- Rolling update no Swarm: docs.docker.com/engine/swarm/swarm-tutorial/rolling-update/
- `docker service update`: docs.docker.com/reference/cli/docker/service/update/
- Visão geral do modo Swarm: docs.docker.com/engine/swarm/